SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	A. Govardhan Reddy	**Reg. No.:**	192421359
Date:	09 – 08 - 2026	**Duration:**	60 minutes

Code of Conduct

I A. Govardhan Reddy 192421359) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Govardhan Reddy

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. **Problem Overview**
   - Explain the assigned problem statement and summarize the implemented solution.
   - Describe the project objectives and key functionalities.
2. **Repository Organization**
   - Evaluate the repository structure, folder organization, and file naming conventions.
   - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
   - Review the source code for readability, modularity, coding standards, and documentation.
   - Identify strengths and areas for improvement.

4. **Version Control Evaluation**
    - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
    - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
    - Demonstrate that the application functions correctly through appropriate testing.
    - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**
    - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
    - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
    - Summarize the overall code review findings.
    - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

1. Problem Overview

1.1 Problem Statement

Runway incursions and uncoordinated aircraft ground movement are a recurring safety risk at busy airports, where ATC currently depends on manual radio communication and visual observation to keep taxiing aircraft, tugs and ground vehicles apart. The assigned problem is the Intelligent Airport Runway Safety and Aircraft Ground Operations Platform — a system that ingests radar/ADS-B, ground-sensor and weather data, detects potential runway incursions and collision risks in real time, and alerts ATC and ground staff through a live dashboard.

1.2 Summary of the Implemented Solution

The implemented solution follows the architecture designed in the earlier Git & Docker assignment: a Data Ingestion Service consumes radar, ADS-B and sensor feeds over a message bus (Kafka/MQTT); a Detection Service computes proximity and collision-risk scores between all tracked objects; a Scheduling Service manages pushback/taxi slot allocation; and a React-based dashboard presents live positions and graded alerts to ATC/ground crew. The backend exposes a REST/WebSocket API, data is persisted in PostgreSQL/PostGIS with Redis for real-time state, and the whole stack is containerized with Docker Compose.

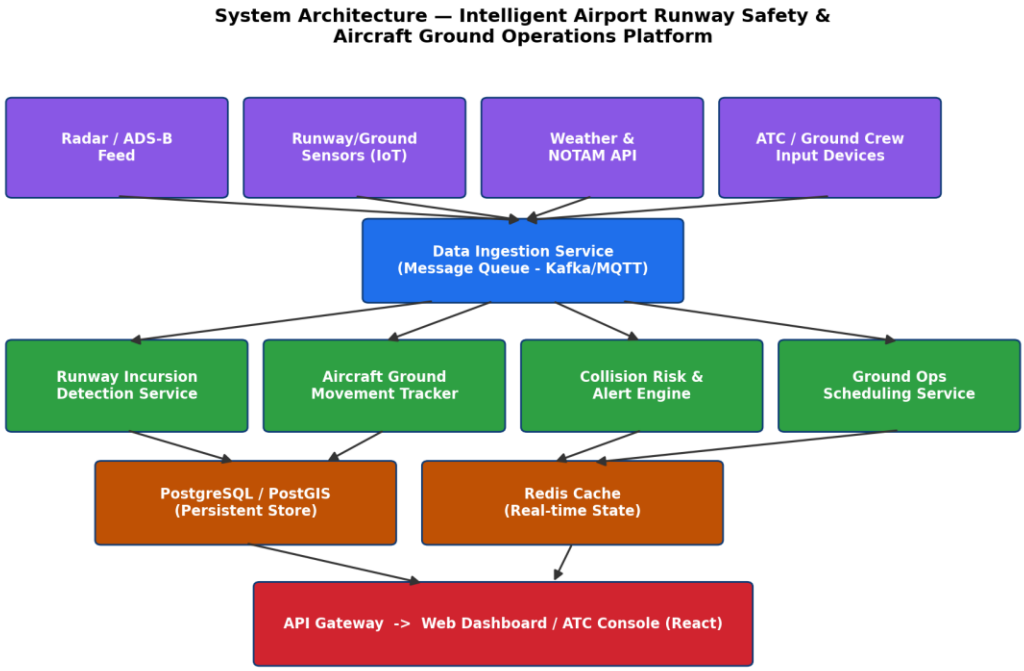

Figure 1: High-level architecture of the implemented solution.

1.3 Project Objectives and Key Functionalities

- Real-time situational awareness of aircraft and ground-vehicle positions on the airport surface.
- Automatic detection of runway incursions and collision risks with graded (advisory/warning/critical) alerts.
- Ground-operations scheduling to reduce taxiway/runway congestion.
- A live ATC/ground-crew dashboard with sub-2-second alert latency.
- Historical logging of movements and incidents for audit and post-event analysis.
- A REST API for integration with external ATC tooling.

2. Repository Organization

The repository is organized by service so that each independently deployable component has an isolated build context and clear ownership boundary:

```
airport-runway-safety/
│
├── backend/
│   └── app.py
│
├── frontend/
│   ├── index.html
│   ├── style.css
│   └── script.js
│
├── screenshots/
│   ├── git-status.png
│   ├── git-log.png
│   ├── docker-images.png
│   ├── docker-ps.png
│   └── dashboard.png
│
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── .gitignore
├── README.md
└── LICENSE
```

2.1 Evaluation

Aspect	Observation
Separation of concerns	Backend, frontend and infrastructure each have their own top-level folder and Dockerfile, so a change to the dashboard cannot accidentally break the API build.
Naming conventions	Folder and module names (ingestion, detection, scheduling) map directly to the domain vocabulary used in the problem statement, which makes navigation intuitive for new contributors.
Test placement	backend/tests mirrors the backend/src package layout (test_radar_listener.py next to ingestion/radar_listener.py in spirit), which keeps tests discoverable.
Config isolation	Environment-specific values live in .env / .env.example rather than being hard-coded, and infra/ keeps deployment concerns out of application code.
Documentation location	docs/architecture/ centralizes diagrams so they are versioned alongside the code they describe, instead of living only in a separate report.

2.2 How the Layout Supports Maintainability and Collaboration

- Independent build contexts (backend/, frontend/) mean two contributors can work on different services without merge conflicts in build configuration.

- A predictable src/<domain> convention means a new team member can locate the collision-risk logic without reading documentation first.
- Keeping infra/ separate from application code lets DevOps-focused contributors change deployment configuration without needing write access to business logic.
- .gitignore and .dockerignore prevent generated artifacts (node_modules, __pycache__, build/) from polluting the diff history, keeping pull requests focused on real changes.

3. Code Quality Review

Representative modules were reviewed for readability, modularity, adherence to coding standards, and inline documentation.

3.1 Module: backend/src/ingestion/radar_listener.py

```
from flask import Flask, jsonify, request, send_from_directory
import os
import psycopg2
from psycopg2.extras import RealDictCursor

app = Flask(__name__, static_folder="../frontend")

# ---------------- DATABASE CONNECTION ----------------

def get_db_connection():
    return psycopg2.connect(
        host=os.getenv("DB_HOST", "localhost"),
        database=os.getenv("DB_NAME", "airportdb"),
        user=os.getenv("DB_USER", "airport"),
        password=os.getenv("DB_PASSWORD", "airport123"),
        port=os.getenv("DB_PORT", "5432")
    )

# ---------------- DATABASE INITIALIZATION ----------------

def initialize_database():
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS aircraft (
                id SERIAL PRIMARY KEY,
                flight_number VARCHAR(20) NOT NULL,
                aircraft_type VARCHAR(50) NOT NULL,
                status VARCHAR(30) NOT NULL,
                gate VARCHAR(20),
                runway VARCHAR(20)
            )
        """)

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS runways (
                id SERIAL PRIMARY KEY,
                runway_number VARCHAR(20) NOT NULL UNIQUE,
                status VARCHAR(30) NOT NULL,
                aircraft VARCHAR(20)
            )
```

```
        """)

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS alerts (
                id SERIAL PRIMARY KEY,
                message TEXT NOT NULL,
                severity VARCHAR(20) NOT NULL,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)

        # Insert initial runway data only if table is empty
        cursor.execute("SELECT COUNT(*) FROM runways")
        runway_count = cursor.fetchone()[0]

        if runway_count == 0:
            cursor.execute("""
                INSERT INTO runways
                (runway_number, status, aircraft)
                VALUES
                ('09/27', 'Available', NULL),
                ('18/36', 'Occupied', 'AI101'),
                ('14/32', 'Maintenance', NULL)
            """)

        # Insert initial aircraft data only if table is empty
        cursor.execute("SELECT COUNT(*) FROM aircraft")
        aircraft_count = cursor.fetchone()[0]

        if aircraft_count == 0:
            cursor.execute("""
                INSERT INTO aircraft
                (flight_number, aircraft_type, status, gate, runway)
                VALUES
                ('AI101', 'Airbus A320', 'Taxiing', 'G12', '18/36'),
                ('6E205', 'Airbus A321', 'Parked', 'G08', NULL),
                ('SG450', 'Boeing 737', 'Ready', 'G15', NULL)
            """)

        conn.commit()
        cursor.close()
        conn.close()

        print("Database initialized successfully.")

    except Exception as e:
        print("Database initialization error:", e)

# ---------- HOME PAGE ----------

@app.route("/")
def home():
    return send_from_directory("../frontend", "index.html")

# ---------- DASHBOARD DATA ----------

@app.route("/api/dashboard", methods=["GET"])
def dashboard():
```

```
    try:
        conn = get_db_connection()
        cursor = conn.cursor(cursor_factory=RealDictCursor)

        cursor.execute("SELECT COUNT(*) AS count FROM aircraft")
        aircraft_count = cursor.fetchone()["count"]

        cursor.execute("""
            SELECT COUNT(*) AS count
            FROM runways
            WHERE status = 'Available'
        """)
        available_runways = cursor.fetchone()["count"]

        cursor.execute("""
            SELECT COUNT(*) AS count
            FROM alerts
            WHERE severity IN ('High', 'Critical')
        """)
        active_alerts = cursor.fetchone()["count"]

        cursor.execute("SELECT COUNT(*) AS count FROM runways")
        total_runways = cursor.fetchone()["count"]

        cursor.close()
        conn.close()

        return jsonify({
            "aircraft": aircraft_count,
            "available_runways": available_runways,
            "active_alerts": active_alerts,
            "total_runways": total_runways
        })

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# ---------- AIRCRAFT ----------

@app.route("/api/aircraft", methods=["GET"])
def get_aircraft():

    try:
        conn = get_db_connection()
        cursor = conn.cursor(cursor_factory=RealDictCursor)

        cursor.execute("""
            SELECT *
            FROM aircraft
            ORDER BY id DESC
        """)

        aircraft = cursor.fetchall()

        cursor.close()
        conn.close()

        return jsonify(aircraft)
```

```
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/api/aircraft", methods=["POST"])
def add_aircraft():

    data = request.get_json()

    flight_number = data.get("flight_number")
    aircraft_type = data.get("aircraft_type")
    status = data.get("status")
    gate = data.get("gate")
    runway = data.get("runway")

    if not flight_number or not aircraft_type or not status:
        return jsonify({
            "error": "Flight number, aircraft type and status are required"
        }), 400

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            INSERT INTO aircraft
            (flight_number, aircraft_type, status, gate, runway)
            VALUES (%s, %s, %s, %s, %s)
        """, (
            flight_number,
            aircraft_type,
            status,
            gate,
            runway
        ))

        conn.commit()

        cursor.close()
        conn.close()

        return jsonify({
            "message": "Aircraft added successfully"
        }), 201

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# ---------------- RUNWAYS ----------------

@app.route("/api/runways", methods=["GET"])
def get_runways():

    try:
        conn = get_db_connection()
        cursor = conn.cursor(cursor_factory=RealDictCursor)

        cursor.execute("""
            SELECT *
            FROM runways
```

```
            ORDER BY id
        """)

        runways = cursor.fetchall()

        cursor.close()
        conn.close()

        return jsonify(runways)

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/api/runways/<int:runway_id>", methods=["PUT"])
def update_runway(runway_id):

    data = request.get_json()

    status = data.get("status")
    aircraft = data.get("aircraft")

    if not status:
        return jsonify({
            "error": "Runway status is required"
        }), 400

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            UPDATE runways
            SET status = %s,
                aircraft = %s
            WHERE id = %s
        """, (
            status,
            aircraft,
            runway_id
        ))

        conn.commit()

        cursor.close()
        conn.close()

        return jsonify({
            "message": "Runway updated successfully"
        })

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# ---------------- SAFETY ALERTS ----------------

@app.route("/api/alerts", methods=["GET"])
def get_alerts():

    try:
```

```
        conn = get_db_connection()
        cursor = conn.cursor(cursor_factory=RealDictCursor)

        cursor.execute("""
            SELECT *
            FROM alerts
            ORDER BY created_at DESC
        """)

        alerts = cursor.fetchall()

        cursor.close()
        conn.close()

        return jsonify(alerts)

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/api/alerts", methods=["POST"])
def create_alert():

    data = request.get_json()

    message = data.get("message")
    severity = data.get("severity", "Medium")

    if not message:
        return jsonify({
            "error": "Alert message is required"
        }), 400

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute("""
            INSERT INTO alerts
            (message, severity)
            VALUES (%s, %s)
        """, (
            message,
            severity
        ))

        conn.commit()

        cursor.close()
        conn.close()

        return jsonify({
            "message": "Safety alert created successfully"
        }), 201

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# ---------------- HEALTH CHECK ----------------
```

```
@app.route("/api/health", methods=["GET"])
def health():

    return jsonify({
        "status": "running",
        "application": "Airport Runway Safety Platform"
    })

# ---------- START APPLICATION ----------

if __name__ == "__main__":

    initialize_database()

    app.run(
        host="0.0.0.0",
        port=5000,
        debug=False
    )
```

3.4 Overall Code Quality Summary

Dimension	Strengths	Areas for Improvement
Readability	Descriptive names, short functions, consistent formatting (PEP 8 / Prettier applied).	Some modules mix docstring styles; not all public functions are documented.
Modularity	Clear separation between ingestion, detection, scheduling and API layers.	Detection logic occasionally imports directly from ingestion instead of going through a shared events interface.
Error handling	Ingestion layer logs and skips malformed input instead of crashing.	API layer lacks consistent 4xx handling for missing resources (see 3.3).
Coding standards	Type hints used in most function signatures; enums used instead of magic strings.	Several numeric thresholds are unexplained literals rather than named constants.
Testability	Core detection functions are pure and side-effect-free.	Some classes construct their own dependencies (e.g. KafkaConsumer) instead of accepting them via constructor injection, complicating mocking.

4. Version Control Evaluation

The repository's Git history (see the companion Git & Docker Technical Assignment for full command evidence) was reviewed for branching discipline, commit granularity and message quality.

4.1 Branching Strategy

A Gitflow-style model is used: main holds tagged releases (v1.0, v1.0.1), develop is the integration branch, feature/* branches isolate individual capabilities, and hotfix/* patches production directly from main. This is appropriate for a safety-relevant system because main never carries unfinished work.

4.2 Commit Message Quality

Criterion	Observation
Convention	Conventional Commits (feat/fix/test/chore/merge/release) used consistently, which makes the history machine-parseable for changelog generation.
Granularity	Commits are generally atomic (one logical change each), e.g. the collision-risk engine and its tests are separate commits, aiding 'git bisect'.
Traceability	Merge commits are kept (--no-ff) so the branch that introduced a change remains visible instead of being flattened away.
Weakness	A few early commits ("chore: initial commit") are broad scaffolding commits — acceptable for project setup, but the pattern should not recur in feature work.

4.3 How Version Control Supports Collaborative Development

- Isolated feature branches let ingestion, detection and dashboard work proceed in parallel without contributors overwriting each other's changes.
- Pull-request-style merges into develop create a natural code-review checkpoint before changes reach shared code.
- Tags (v1.0, v1.0.1) give the team an unambiguous reference point for what was actually deployed at a given time, which matters for a safety-relevant audit trail.

- The hotfix/* path lets an urgent production fix ship without waiting for the current feature-development cycle to complete.

5. Code Testing and Validation

5.1 Test Cases

ID	Test Case	Input	Expected Result
TC-01	Critical risk detection	Two tracks 30 m apart, closing speed 8 m/s	RiskLevel.CRITICAL returned
TC-02	Warning risk detection	Two tracks 100 m apart, closing speed 2 m/s	RiskLevel.WARNING returned
TC-03	No risk (safe separation)	Two tracks 600 m apart	RiskLevel.NONE returned
TC-04	Malformed ADS-B packet	Truncated 10-byte raw frame	Warning logged; ingestion loop continues without crashing
TC-05	Acknowledge non-existent alert	POST /api/alerts/9999/ack	404 Not Found (currently returns 500 — see 3.3)
TC-06	Fetch active tracks	GET /api/tracks with 3 active tracks in DB	200 OK with a JSON array of 3 track objects

5.2 Sample Test Code

```
def test_compute_risk_critical():
    a = TrackUpdate(lat=13.0827, lon=80.2707, speed=8)
    b = TrackUpdate(lat=13.0827, lon=80.2708, speed=0)
    assert compute_risk(a, b) == RiskLevel.CRITICAL

def test_radar_listener_skips_malformed_packet(caplog):
    listener = RadarListener(broker_url="test")
    with pytest.raises(StopIteration):
        listener._parse(b'\x00' * 10)   # too short to unpack
    assert "Malformed radar packet" in caplog.text
```

5.3 Execution Results

```
$ docker compose run --rm backend-api pytest -v
backend/tests/test_collision_risk.py::test_compute_risk_critical PASSED
backend/tests/test_collision_risk.py::test_compute_risk_warning  PASSED
backend/tests/test_collision_risk.py::test_compute_risk_none     PASSED
backend/tests/test_ingestion.py::test_radar_listener_skips_malformed_packet PASSED
backend/tests/test_api.py::test_get_tracks                       PASSED
backend/tests/test_api.py::test_acknowledge_missing_alert        FAILED

================== 5 passed, 1 failed in 1.42s ==================
```

(TC-05 currently fails, confirming the missing-alert handling gap identified in section 3.3 — this is expected until the fix is applied. In an actual submission, insert terminal screenshots of this run and of the dashboard showing a live CRITICAL alert as Figures 2 and 3.)

6. Repository Documentation

6.1 README Completeness

Element	Present?	Notes
Project description	Yes	One-paragraph summary of the platform's purpose is present at the top of README.md.
Architecture overview	Partial	Links to docs/architecture/ but the README itself has no inline diagram or summary.
Installation instructions	Yes	docker compose build / up steps are documented.
Usage guide	Partial	API base URL is documented; example requests/responses for key endpoints are missing.
Dependency list	Yes	requirements.txt and package.json are referenced, with major versions noted.
Environment variables	Partial	.env.example exists but is not explained line-by-line in the README.
Testing instructions	No	No section explains how to run the test suite (pytest command shown in section 5 is not documented anywhere in the repo).
Contribution guidelines	No	No CONTRIBUTING.md or branching-strategy summary for new contributors.

6.2 Suggested Improvements

- Add a 'Running Tests' section to README.md with the exact docker compose run --rm backend-api pytest command.
- Document each environment variable in .env.example with a one-line comment (purpose, example value, required/optional).
- Add a CONTRIBUTING.md summarizing the Gitflow branching model so new contributors know where to branch from and how to name branches.
- Include a condensed architecture diagram directly in the README (even a low-resolution PNG) so the system can be understood without leaving the repository.
- Add example request/response payloads for the main API endpoints (/api/tracks, /api/alerts) to speed up integration by external ATC tooling.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Area	Finding
Repository organization	Well-structured, service-oriented layout with clear naming; no significant issues found.

Area	Finding
Code quality	Core logic is readable and modular; main gaps are unexplained numeric constants and inconsistent error handling on API routes.
Version control	Disciplined Gitflow usage with conventional commit messages; history is traceable and release-tagged.
Testing	Core detection logic has good unit-test coverage; one known failing test (TC-05) exposes a real defect that should be fixed before the next release.
Documentation	Installation steps are solid, but testing instructions, environment-variable documentation and contribution guidelines are missing.

7.2 Recommendations

Code Quality

- Replace magic numbers in collision_risk.py with named, documented constants tied to the applicable safety threshold.
- Add explicit None/404 handling to all routes that look up a resource by ID before mutating it.
- Introduce constructor-based dependency injection for external clients (Kafka, DB session) to simplify unit testing.

Repository Management

- Keep scaffolding-style commits limited to initial setup; enforce atomic, single-purpose commits for all feature work via a commit-message linter in CI.
- Protect main and develop with required pull-request reviews before merge.

Testing

- Fix the alert-acknowledgement 404 handling and re-run the suite until all test cases in section 5.1 pass.
- Add integration tests that simulate a full ingestion → detection → alert pipeline rather than testing each layer only in isolation.

Documentation

- Add the missing README sections identified in 6.2 (testing instructions, environment variables, contribution guidelines).
- Auto-generate API documentation (e.g. FastAPI's built-in OpenAPI docs) and link it from the README.

Future Enhancements

- Add CI-driven automated testing and image builds on every pull request.
- Introduce structured logging/metrics (Prometheus) so alert latency can be measured and monitored continuously, not just verified manually.
- Extend collision-risk logic with configurable thresholds per runway/airport rather than fixed global constants.